

Naravno. Evo tog dela napisanog **redosledom po svakom od 4 zahteva iz 2. grupe**, baš kao što ti treba — znači: **šta je promenjeno, u kojim fajlovima i zašto**.

2) Unapređenje postojećih funkcija

2.1. Na stranici „Moji projekti“ dodati taster „Započni“ uz svaki projekat koji je „U pripremi“, što prebacuje projekat u stanje „U realizaciji“ i čuva izmenu u bazi podataka.

Šta sam radio

Za ovaj zahtev sam dodao mogućnost da se projekat iz stanja **U pripremi** prebaci u **U realizaciji** direktno sa stranice „Moji projekti“.

Izmene na serveru

Exam.App/Services/IProjectService.cs

U interfejs sam dodao novu metodu:

```
Task<ProjectDto> StartAsync(int id, string username);
```

To je bilo potrebno da servisni sloj dobije jasno definisanu operaciju za pokretanje projekta.

Exam.App/Services/ProjectService.cs

U servis sam dodao metodu `StartAsync`.

Ona radi sledeće:

- pronalazi projekat koji pripada prijavljenom korisniku,
- proverava da li je trenutni status `Draft` odnosno „U pripremi“,
- ako jeste, menja status u `Published` odnosno „U realizaciji“,
- čuva promenu u bazi.

Dodao sam i pomoćnu metodu `GetOwnedProjectAsync`, da se ne ponavlja ista logika provere:

- da li je korisnik prijavljen,
- da li projekat postoji,
- da li projekat pripada baš tom korisniku.

To je važno da korisnik ne bi mogao da menja tuđe projekte.

Exam.App/Controllers/ProjectController.cs

U kontroleru sam dodao novu rutu:

```
[HttpPost("{id}/start")]
```

Ona poziva servisnu metodu `StartAsync`.

To znači da frontend sada može da pošalje zahtev tipa:

```
POST /api/projects/{id}/start
```

i projekat će biti prebačen u naredni status.

Izmene na klijentu

src/features/projects/services/projectService.js

Dodao sam novu funkciju:

```
export const startProject = async (id) => {  
  const response = await axiosConfig.post(`/projects/${id}/start`);  
  return response.data;  
};
```

Ona služi da frontend može da pozove novi backend endpoint.

src/features/projects/ProjectList.jsx

U listi projekata sam dodao prikaz dugmeta **Započni** samo za projekte čiji je status 0 odnosno „U pripremi“.

Takođe sam dodao logiku da se dugmad prikazuju uslovno, u zavisnosti od statusa projekta.

Zašto je to urađeno

Zahtev traži da korisnik može jednim klikom da pokrene projekat i da se to odmah sačuva u bazi. Zato je bilo potrebno:

- nova serverska akcija,
- nova API ruta,
- nova frontend funkcija,
- i dugme na listi projekata.

2.2. Na istoj stranici dodati taster „Zaključ“ uz svaki projekat koji je „U realizaciji“, što postavlja vreme

kompletiranja projekta na trenutni momenat i ažurira status. Kompletiranim projektima nije moguće menjati naziv, opis i datum početka i ovo treba podržati na klijentu i serveru. Kompletirani projekat se može vratiti u stanje „U pripremi“, čime se uklanja vreme kompletiranja.

Šta sam radio

Ovo je bio opširniji zahtev, pa sam ga rešavao u dva dela:

1. zaključenje projekta,
 2. zabrana izmene kompletiranog projekta i vraćanje u pripremu.
-

A) Zaključenje projekta

Exam.App/Services/IProjectService.cs

Dodao sam:

```
Task<ProjectDto> ConcludeAsync(int id, string username);  
Task<ProjectDto> ReopenAsync(int id, string username);
```

Exam.App/Services/ProjectService.cs

Dodao sam metodu `ConcludeAsync`.

Ona:

- proverava da projekat pripada prijavljenom korisniku,
- proverava da li je projekat u statusu `Published` odnosno „U realizaciji“,
- menja status u `Completed` odnosno „Kompletiran“,
- upisuje trenutno vreme u `CompletedAt`,
- čuva promenu.

Dodao sam i `ReopenAsync`.

Ona:

- dozvoljava vraćanje samo iz stanja `Completed`,
- menja status nazad u `Draft`,
- briše `CompletedAt`.

Time je ispunjen i deo da se kompletiran projekat može vratiti u prethodno stanje.

Exam.App/Controllers/ProjectController.cs

Dodao sam dve nove rute:

```
[HttpPost("{id}/conclude")]
[HttpPost("{id}/reopen")]
```

One povezuju frontend i servisni sloj sa novim akcijama.

src/features/projects/services/projectService.js

Dodao sam:

```
export const concludeProject = async (id) => { ... }
export const reopenProject = async (id) => { ... }
```

src/features/projects/ProjectList.jsx

Dodao sam dugmad:

- **Zaključ**i za projekte u statusu 1 / Published,
- **Vrati u pripremu** za projekte u statusu 2 / Completed.

Tako korisnik može da upravlja statusima direktno iz liste.

B) Zabrana izmene kompletiranog projekta

Exam.App/Services/ProjectService.cs

U `UpdateAsync` sam dodao posebnu proveru za projekat koji je već kompletiran.

Ako je projekat u statusu `Completed`, tada:

- naziv, opis i datum početka ne smeju da se menjaju,
- ako se pokuša promena, baca se greška.

To je urađeno na serveru jer server mora da bude poslednja linija zaštite.

src/features/projects/MyProjects.jsx

Na klijentu sam dodao logiku da se, kada je projekat kompletiran:

- polja forme onemoguće,
- dugme „Sačuvaj“ sakrije,

- korisnik dobije poruku da projekat nije moguće menjati dok je kompletiran.

To je urađeno da bi UI pratio pravilo iz servera i da korisnik odmah vidi da izmena nije dozvoljena.

Zašto je to urađeno

Zahtev je tražio da:

- kompletirani projekti ne budu slučajno menjani,
- zaključenje projekta upiše vreme završetka,
- i da postoji povratak u pripremu.

Zato sam izmene rasporedio i na backend i na frontend:

- backend radi stvarnu kontrolu i čuva stanje u bazi,
 - frontend samo vizuelno podržava tu logiku.
-

2.3. Na stranici „Korisnici“ treba za svakog korisnika, uz ime i prezime, prikazati broj kompletiranih projekata, broj projekata u realizaciji i datum najskorije kompletiranog projekta. Do klijenta treba da stigne minimalni skup podataka za prikaz ovih informacija, kako bi se smanjila veličina HTTP odgovora.

Šta sam radio

Ovaj zahtev sam rešio tako da server ne šalje ceo objekat korisnika, nego samo ono što je potrebno za prikaz.

Izmene na serveru

`Exam.App/Services/Dtos/UserSummaryDto.cs`

Napravio sam novi DTO:

- Id
- Name
- Surname

- CompletedProjectsCount
- InProgressProjectsCount
- LastCompletedAt

To je minimalni skup podataka potreban za prikaz na stranici „Korisnici“.

Exam.App/Infrastructure/Database/Repositories/UserRepository.cs

U repozitorijumu sam promenio način učitavanja korisnika.

Sada:

- uzimam samo korisnike sa ulogom `User`,
- za svakog korisnika računam:
 - koliko ima kompletiranih projekata,
 - koliko ima projekata u realizaciji,
 - kada mu je bio poslednji kompletirani projekat,
- i vraćam upravo DTO, ne ceo entitet.

To znači da API ne šalje nepotrebne podatke.

Exam.App/Services/Dtos/UserSummaryDto.cs

Ovaj DTO je važan baš zbog ovog zahteva, jer omogućava da odgovor servera bude mali i precizan.

Izmene na klijentu

src/features/users/UserList.jsx

U listi korisnika sam dodao prikaz:

- imena i prezimena,
- broja kompletiranih projekata,
- broja projekata u realizaciji,
- datuma poslednjeg kompletiranja.

Tako korisnik na stranici odmah vidi tražene informacije.

Zašto je to urađeno

Zadatak traži minimalan skup podataka iz razloga performansi. Zato:

- server ne šalje nepotrebne entitete,
 - frontend prikazuje samo što mu je potrebno.
-

2.4. Na istoj stranici sortirati spisak spram broja kompletiranih projekata, pa zatim spram broja projekata u realizaciji tako da najvredniji korisnik bude na vrhu spiska. Pri tom, uvažiti paginaciju (prva stranica će imati najaktivnije korisnike, a poslednja one bez projekata).

Šta sam radio

Ovaj zahtev sam rešio na serveru, jer je važno da se **sortiranje** desi **pre** **paginacije**.

Izmene na serveru

`Exam.App/Infrastructure/Database/Repositories/UserRepository.cs`

U upitu sam dodao sortiranje:

1. prvo po broju kompletiranih projekata, opadajuće,
2. zatim po broju projekata u realizaciji, opadajuće,
3. zatim po imenu i prezimenu kao dodatni kriterijum.

To znači da korisnici sa najviše uspešno završenih projekata dolaze prvi.

Nakon toga se radi:

- `Skip(...)`
- `Take(...)`

što znači da se tek posle sortiranja primenjuje paginacija.

Izmene na klijentu

`src/features/users/UserList.jsx`

Na klijentu sam već postojao mehanizam za:

- trenutnu stranicu,
- veličinu stranice,
- prikaz ukupnog broja stranica.

Pošto server vraća već sortirane podatke, frontend samo prikazuje rezultat i menja strane.

Zašto je to urađeno

Da sam sortirao tek posle paginacije, rezultat ne bi bio ispravan — korisnici sa najviše projekata možda ne bi bili na prvoj strani.

Zato je redosled:

1. sortiranje,
2. pa tek onda paginacija.

Ako želiš, odmah mogu da nastavim sa **3) Uvođenje veština** istim stilom: prvo kompletni fajlovi, pa zatim ovakvo objašnjenje po svakom zahtevu redom.